# XPU technology for fast and efficient FDTD Simulations using modern CPUs Cache Memory Bandwidth

A. Lauer[1], W. Simon[1], A. Wien[1],
[1]IMST GmbH, D–47475 Kamp-Lintfort, Germany, e-mail lauer@imst.de

***Abstract*—EM Simulations play a key role in the design of antennas and RF components. Due to increasing complexity, higher package density or taking into account environmental aspects, the simulation effort is steadily rising, so simulation acceleration techniques become more and more important. Solutions based on graphic cards (GPU) are expensive and limited in usable memory. This paper presents the XPU technology which is a smart implementation of the Finite Difference Time Domain (FDTD) algorithm on modern CPU architectures. It can drastically reduce simulation times while (in contrast to GPU solutions) maintaining full access to the available main memory. Simulations of a 77 GHz radar sensor integrated into a car as well as a wireless vehicle charging system simulation show how the XPU technique is applied to state-of-the-art RF systems.**

***Index Terms*—EM simulation, acceleration techniques, FDTD, Computer Technology, GPU, Antennas, RF components, Radar, 77 GHz, Wireless charging.**

## I. Introduction

The well known classical FDTD method [1], [2], [3], [4] keeps samples of three electric field components $E_x$,$E_y$,$E_z$ as well as samples of three magnetic field components $H_x$,$H_y$,$H_z$ in computer memory. The fields are calculated in dependence of time[1], overwriting the complete E-Field rsp. the complete H-Field with new samples in one atomic step,

$$\vec{H}(\vec{x},t+1) = \alpha_{\mathrm{H}}(\vec{x})\,\vec{H}(\vec{x},t) + \sum_{\vec{x}_1} \nabla^{\times}_{\mathrm{H}}(\vec{x},\vec{x}_1)\,\vec{E}(\vec{x}_1,t), \qquad (1)$$

$$\vec{E}(\vec{x},t+1) = \alpha_{\mathrm{E}}(\vec{x})\,\vec{E}(\vec{x},t) + \sum_{\vec{x}_1} \nabla^{\times}_{\mathrm{E}}(\vec{x},\vec{x}_1)\,\vec{H}(\vec{x}_1,t+1) \qquad (2)$$

where $\vec{x} = (x,y,z)^T$ denotes vector of space indices into the discrete fields, $t$ is the discrete time index, $\alpha_{\mathrm{H}}$ and $\alpha_{\mathrm{E}}$ are space-dependent constants, $\nabla^{\times}_{\mathrm{H}}$ and $\nabla^{\times}_{\mathrm{E}}$ are the discrete equivalent of the curl operators in Maxwell's equations multiplied by space-dependent material parameters [5].

Lossless media are assumed [2] ($\alpha_{\mathrm{H}} \equiv 1$,$\alpha_{\mathrm{E}} \equiv 1$), $\nabla^{\times}_{\mathrm{H}}$ and $\nabla^{\times}_{\mathrm{E}}$ are sparse space-dependent matrices with only four nonzero elements per row, which are pairwise equal.

The structure of (1,2) permits to overwrite the values $\vec{H}(\vec{x},t=n)$ in RAM with those of $\vec{H}(\vec{x},t=n+1)$, because $\vec{H}(\vec{x},t=n)$ is not needed any more after (1) is done by for a single space index. For the same reason $\vec{E}(\vec{x},t=n+1)$ can replace $\vec{E}(\vec{x},t=n)$ in computer memory.

Without further optimization we need four floating-point additions (three for the curl, one for the + in (1) rsp. (2)) and two multiplications per field component[3], altogether six Floating-point operations per component.

An optimistic assumption [4] regarding the computer memory bandwidth required is that for the computation of (1) for all space indices the complete E-Field and the complete H-Field has to be read from RAM, the complete E-Field has to be written back, summing up to three floating-point RAM operations per component update. Assuming single precision floating point numbers (four Bytes each), this means six Components $\times$ three Operations x four Bytes = 72 Bytes per cell.

Let's assume a state of the art Processor (Intel Core I7 5960X) with eight processor cores, 3.0 GHz Clock Speed and four DDR4 RAM Channels at 2100 MHz[6]. The max. RAM Memory Bandwidth is 68 GBytes/s limiting the FDTD Speed to 944 MCells/s (Million Cells per second and time step).

The mere calculation power of the CPU is much more impressive. Usage of AVX2[7] command extension of x86 CPUs permits to do eight floating point operations per cycle and Core in parallel, this is approx. 200 GFlops/s (Billion floating point operations per second) for eight cores. Because additions and multiplications can be done together in one clock cycle (FMA3[8]), only the four additions per field component are counted here. Per unit cell 4x6=24 operations have to be done, limiting the FDTD speed to approx. 8300 MCells/s.

This simple calculation shows the bottleneck: Memory

[1]and space

[2]For better understanding. The application examples are calculated including loss.

[3]due to the pairwise equality of the four non-zero elements in one row of $\nabla^{\times}_{\mathrm{H}}(\vec{x},\vec{x}_1)$ rsp. $\nabla^{\times}_{\mathrm{E}}(\vec{x},\vec{x}_1)$

[4]Assuming the CPU Cache is big, fast and smart enough to avoid RAM access to all data recently used. Furthermore it is assumed that due to efficient storage technique of coefficients access to $\alpha_{\mathrm{H}}$ and $\nabla^{\times}_{\mathrm{H}}$ can be neglected here.

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:27 UTC from IEEE Xplore. Restrictions apply.

access slows down FDTD speed by a factor of more than eight.

The computer's Cache Memory is much faster. The Intel Core I7 5960X offers a 20 MByte L3-Cache with a ring-bus architecture [9] permitting up to 32 Bytes per CPU cycle and core to be read or written. This results in a total L3 Bandwidth of 768 GBytes per second, limiting FDTD speed to 10.6 GCells/s.

So if (1,2) can be done from Cache Memory, FDTD will speed up a lot.

Section II sketches, how to do in a first step the $\vec{E}$ and $\vec{H}$ updates together, witch is the preparing step to be able to keep multiple time steps in computer memory for efficient FDTD simulation as written in section III.

## II. Updating $\vec{E}$ and $\vec{H}$ together

The further considerations are simplified if the Field is organized in Cells. Six field components are taken as a unit cell as sketched in Fig. 1.

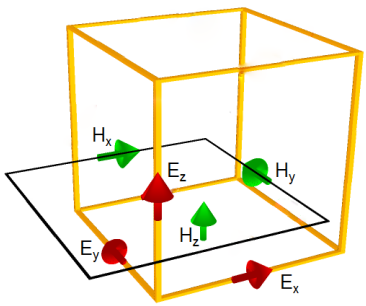


Fig. 1. The unit cell with associated field components.

Furthermore it is assumed, that the calculation loops for the field updates go through the fields step by step in positive coordinate direction. It does not matter here, but later the inner loop of the program under consideration will be over $x$, the middle over $z$ and the outer over $y$.

After the $\vec{H}(\vec{x},t)$ field of one unit cell is updated to $\vec{H}(\vec{x},t+1)$, the unit cell's $\vec{E}(\vec{x},t)$ field components are not needed for any further calculations and can be *immediately* updated to $\vec{E}(\vec{x},t+1)$. This is true due to the position of the non-zero elements in the discrete curl operators in (1,2).

So instead of doing atomic half time steps as in (1,2), we are able now to update the $\vec{E}$ and $\vec{H}$ field in the same program loop doing a full time step.

The merging of the $\vec{E}$ and $\vec{H}$ updating procedure reduces memory access time already (reduces RAM access to 48 Bytes per cell in the consideration above, raising the memory bandwidth FDTD limit to approx. 1400 MCells/s). Furthermore it is the basis of the multi-time stepping considerations in the next section.

## III. Multiple time stepping

With the optimizations of section II one complete time step is done by looping through (1,2). Under optimistic assumptions like above, this requires to read the complete field ($\vec{E}$ and $\vec{H}$ for all space indices) from RAM and write it back to RAM to do one complete time step. In this section, it is investigated, how multiple time steps can be done with *one* RAM field read and write.

For simplicity it is assumed here, that one $x$-row, defined as all unit cells with fixed space indices $y$ and $z$, is updated atomically from $t$ to $t+1$.

Putting together one $x$-row of $\vec{E}$ and $\vec{H}$ Field Components in a single Vector $\vec{F}(y,z,t)$ we obtain from (1,2)

$$\begin{aligned}\vec{F}(y,z,t+1) = {} & C_0(y,z)\,\vec{F}(y,z,t) + C_{y+}(y,z)\,\vec{F}(y+1,z,t) \\ & + C_{y-}(y,z)\,\vec{F}(y-1,z,t+1) \\ & + C_{z+}\,\vec{F}(y,z+1,t) + C_{z-}\,\vec{F}(y,z-1,t+1),\end{aligned} \tag{3}$$

where the $C_\nu$ denote sparse coefficient Matrices doing operations in $x$-direction which are not of interest in detail here.

Here a short explanation of the time-indices used in (3). The unit cell in Fig. 1 is chosen in a way, that only $\vec{H}$-components of $\vec{F}(y,z-1,t+1)$ and $\vec{F}(y-1,z,t+1)$ are visible to the update equations of $\vec{F}(y,z,t)$, thus $t+1$ is the right time index according to the sum in (2). In the same way, only $\vec{E}$-components of $\vec{F}(y,z+1,t)$ and $\vec{F}(y+1,z,t)$ are visible to the update equations of $\vec{F}(y,z,t)$, here $t$ is the right time index according to (1).

Equation (3) is the basis to decide, if an $x$-row $\vec{F}(y,z,t)$ can be updated. This is exactly the case, when the four neighbor-rows in (3) are already updated to the proper time index, thus (3) is the starting point to design cache optimized FDTD strategies.

Before a simple example is given, that shows how (3) is used in practical cases, some conventions are given, also valid for more examples coming:

1) Indices $y$ and $z$ count starting from zero up to $y_{\max}$ and $z_{\max}$.
2) Implicitly $\vec{F}(y,z,t)=0$, if $y<0$, $y>y_{\max}$, $z<0$ or $z>z_{\max}$ for all times.
3) The scenario starts at $t=0$.

At the start, all $x$-rows are at $t=0$. Lets now investigate, if it is permitted (3) to step $\vec{F}(0,0,0)$ to $\vec{F}(0,0,1)$. The rows $\vec{F}(y,z,t)$, $\vec{F}(y+1,z,t)$ and $\vec{F}(y,z+1,t)$ are OK, all at $t=0$.

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:27 UTC from IEEE Xplore. Restrictions apply.

The rows $\vec{F}(y-1,z,t+1)$, $\vec{F}(y,z-1,t+1)$ are out of area and always zero, also OK to perform (3). That means, it is permitted to do the update $\vec{F}(0,0,0)$ to $\vec{F}(0,0,1)$.

Checking if an immediate update $\vec{F}(0,0,1)$ to $\vec{F}(0,0,2)$ fails, because $\vec{F}(y+1,z,t)$ and $\vec{F}(y,z+1,t)$ are still at $t=0$. But once they are updated ($\vec{F}(1,0,0)$ to $\vec{F}(1,0,1)$ and $\vec{F}(0,1,0)$ to $\vec{F}(0,1,1)$, updating $\vec{F}(0,0,1)$ to $\vec{F}(0,0,2)$ is possible as the next step.

This means, that already after three $x$-row updates from $t=0$ to $t=1$, one $x$-row can be updated again from $t=1$ to $t=2$.

Since $x$-rows are usually small with regard the size of the CPU's L3 Cache (e.g. 200 cells in one $x$-row means 4800 Bytes, a 20 MBytes L3 cache can hold thousands of $x$-rows), it is possible to construct FDTD strategies that can e.g. do 24 time steps with one RAM walk. This pushes the RAM FDTD speed limit considered above to 24x1400 MCells/s = 33000 MCells/s, above the floating point calculation limit.

Furthermore it is easy to nicely spread the multiple time steps among all CPU cores available. When doing a 24 multi-step with 8 CPU cores, core 1 will do all $t=0$ up to $t=3$ updates, core 2 all $t=3$ to $t=6$ updates etc.

## IV. Application Examples

The following challenging simulation examples show the usage of the new XPU technology within the 3D EM simulation software EMPIRE XPU[10]. The first example is an automotive radar application. A 77 GHz radar sensor which is integrated into the bumper of a car is characterized with full wave 3D FDTD simulations. The second example investigates the electromagnetic exposure of wireless vehicle charging systems on human bodies.

### A. 77 GHz Automotive Radar Antenna

Modern cars include more and more radar sensors to increase safety. They determine distance, speed and direction of moving or stationary objects, including people. Designing antennas for such systems is quite challenging. These radar systems are operated at 24 GHz rsp. 77 GHz. They are usually integrated behind the bumper of the car, which normally yields poor antenna performance. Accurate 3D EM simulations of the antennas including the car environment are a must for antenna optimization within a short product development cycle. Due to the memory and speed efficient XPU algorithm of EMPIRE, even 77 GHz antenna systems can be simulated including the car environment as shown in Fig. 2. This simulation model of the 77 GHz radar antenna integrated into the bumper of an Audi A6 car includes the detailed antenna model, the bumper and a part of the rear of the car. A graded mesh is used to be able to resolve the antenna details and the environment. The smallest mesh cell is 30 $\mu$m and the largest mesh cell is 300 $\mu$m. An amount of $576 \times 1601 \times 2381 = 2200$ million FDTD cells is needed to create an accurate mesh for the complete model. This results into 51 GB required RAM memory, a simulation speed of 4100 Million cells per second and a total simulation time including near-field and far-field recording of 45 minutes. A workstation with two Xeon E5-2690 CPU's (2.9 GHz) was used to perform the simulation.

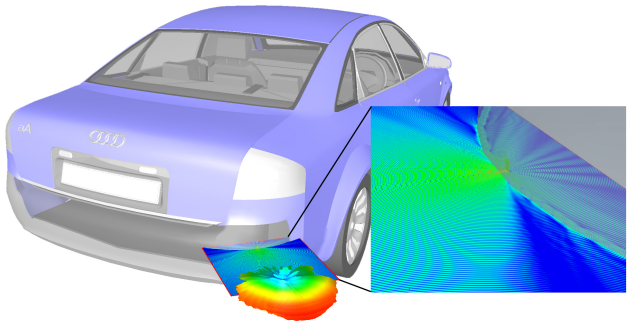

Fig. 2. 77 GHz automotive radar antenna integrated into car bumper

This machine has 16 cores which share the L3 cache. 2.5 MByte of this cache can be used per core, so an optimum of $\frac{2.5\text{MBytes}}{24\text{Bytes}\times 576} = 180$ x-lines should be obtained. Figure 3 shows the observed speed for different multiple time steps per core.

| Multi step per core $n$ | L3 x-lines $4(n+1)^2$ | Speed MCells/s |
|---|---|---|
| 1 | 16 | 1607 |
| 2 | 36 | 2802 |
| 3 | 64 | 3364 |
| 4 | 100 | 3998 |
| 5 | 144 | 4547 |
| 6 | 196 | 4653 |
| 7 | 256 | 4596 |

Fig. 3. Simulation details of the radar antenna. Internal algorithmic speed in dependence of the time stepping.

These simulations can even be performed more efficiently in the latest release by using a new field source definition. This field source can replace the detailed antenna model with feed network and casing by using accurate field data from measurements or previous antenna simulations. By using a local field interpolation, it is possible to model the source with a coarser mesh in the subsequent simulation run including the whole environment, thus reducing the overall required memory and increasing the simulation speed.

### B. Wireless vehicle charging

As electric vehicles are becoming more and more popular, the use of wireless battery charging systems for electric

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:27 UTC from IEEE Xplore. Restrictions apply.

rsp. hybrid vehicles will increase. This technology transmits electrical power by using magnetic resonances that result from changes in magnetic field intensity between a coil positioned on/in the ground below the car, and a receiving coil built into the chassis of the vehicle.

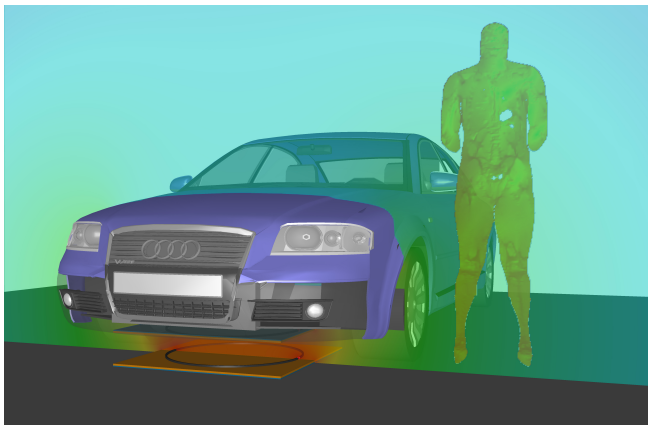

Fig. 4. Wireless charging system: E-field at 1 MHz, averaged induced in body electric field at 90 kHz

As the charging time should be minimal, high magnetic fields are used. This makes it mandatory to investigate the electromagnetic exposure on human beings which are in the vicinity of wireless charging systems of cars. These car charging systems work at e.g. 90 kHz. The recent ICNIRP standard requires the evaluation of the averaged electric field induced inside the body and its distribution in body tissues. EMPIRE can calculate these fields inside the human body by modeling the complete wireless charging system and the environment (see Fig. 4). This EM simulation model considers the ground below the car with the primary charging coil, the secondary coil below the car, the complete car model and the human being which is positioned standing next to the car. An amount of $568 \times 483 \times 902 = 250$ million FDTD cells is needed to create an accurate mesh. This results into 6.5 GB required RAM memory, a simulation speed of 4800 Million cells per second and a total simulation time including near-field and far-field recording of 80 minutes. A workstation with two Xeon E5-2697 CPU's (2.6 GHz) was used to perform the simulation.

This machine has 28 cores which share the L3 cache. 2.5 MByte of this cache can be used per core, so an optimum of $\frac{2.5\text{MBytes}}{24\text{Bytes}\times 568} = 183$ x-lines should be obtained. Figure 5 shows the observed speed for different multiple time steps per core.

An alternative method to access the electromagnetic exposure on the human body is to use a field source, based on measured data, which excites the same field. In addition to the reduced problem size, using a field source based on measured data has the advantage that not the complete wireless charging system details have to be known.

| Multi step per core $n$ | L3 x-lines $6(n+1)^2$ | Speed MCells/s |
|---|---|---|
| 1 | 24 | 1901 |
| 2 | 54 | 3167 |
| 3 | 96 | 4427 |
| 4 | 150 | 5087 |
| 5 | 215 | 5065 |

Fig. 5. Simulation details of the wireless charging system. Internal algorithmic speed in dependence of the time stepping.

## V. Conclusion

The presented XPU technique enables fast and efficient FDTD simulations on modern CPU's. A multiple time stepping algorithm allows to calculate multiple FDTD time steps in the cache memory of the CPU. This increases the simulation speed drastically as the simulation speed is no longer limited by the main memory interface. The whole RAM memory of the PC is fast accessible for the EM simulation.
The XPU technique is applied to simulations of an integrated 77 GHZ radar module and a wireless vehicle charging system. Such applications requiring a large amount of RAM memory can be easily simulated with the XPU technique on consumer PC's and workstations while a calculation on a GPU card would fail due to limited available memory.

## References


[1] K. S. Yee, "Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media," *IEEE Trans. Antennas Propagat.*, vol. 14, pp. 302–307, Mar. 1966.

[2] A. Bahr, A. Lauer, and I. Wolff, "An efficient source formulation for the analysis of microwave circuits using the FDTD," in *IEEE Antennas and Propagat. Soc. Int. Symp.*, vol. 1, (Newport Beach, CA), pp. 244–247, June 1995.

[3] A. Bahr, A. Lauer, and I. Wolff, "Application of the PML absorbing boundary condition to the FDTD analysis of microwave circuits," in *IEEE MTT-S Int. Microwave Symp.*, vol. 1, (Orlando, FL), pp. 27–30, 1995.

[4] A. Lauer and I. Wolff, "Stable and efficient ABCs for graded mesh FDTD," in *IEEE MTT-S Int. Microwave Symp.*, vol. 2, (Baltimore, MD), pp. 461–464, June 1998.

[5] C. C. A. Rennings, A. Lauer and I. Wolff, "Equivalent circuit (ec) fdtd method for dispersive materials: derivation, stability criteria and application examples," in *Time Domain Methods in Electrodynamics, Springer Proceedings in Physics*, vol. 121, (Berlin), pp. 211–238, 2008.

[6] Wikipedia, "Haswell (microarchitecture)," 2014. [Online; accessed 24-Sept-2014].

[7] Wikipedia, "Advanced vector extensions," 2014. [Online; accessed 24-Sept-2014].

[8] Wikipedia, "Fma x86," 2014. [Online; accessed 24-Sept-2014].

[9] R. W. Technologies, "Intels haswell cpu microarchitecture," 2014. [Online; accessed 24-Sept-2014].

[10] A. Wien, "Empire xpu manual," 2014. [Online; http://www.empire.de/EMPIRE-Manual.pdf.


Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:27 UTC from IEEE Xplore. Restrictions apply.